

KỊCH BẢN THUYẾT TRÌNH: CƠ SỞ LÝ THUYẾT & KIẾN TRÚC HỆ THỐNG

Tài liệu này tổng hợp ngắn gọn các điểm chính yếu và vị trí code tương ứng trong dự án **MotoShop** để bạn dễ dàng trình chiếu và thuyết trình.

PHẦN 1: CƠ SỞ LÝ THUYẾT (Lướt nhanh)

Ở phần này, bạn chỉ cần giới thiệu tóm tắt các công nghệ và khái niệm đã sử dụng:

- Nền tảng & Ngôn ngữ:** Dùng Android Studio, code logic bằng Java và code giao diện bằng XML.
- Thành phần giao diện:** Sử dụng các thẻ cơ bản (TextView, EditText, Button). **Điểm nhấn:** Dùng **RecyclerView** kết hợp với Custom Layout (XML riêng cho từng dòng) để hiển thị danh sách tối ưu hiệu năng.
- Thành phần Android:**
 - Activity/Fragment:** Là các màn hình tương tác.
 - Intent:** Chuyển màn hình và truyền dữ liệu (ví dụ: bấm vào một xe, truyền ID của xe đó sang màn hình chi tiết).
 - Adapter:** Cầu nối trung gian để gắn dữ liệu vào **RecyclerView**.
- Lưu trữ & AI:** Dùng Database để lưu trữ dữ liệu (xe, khách, đơn hàng) và tích hợp API Trí tuệ nhân tạo (Gemini) để làm tính năng Chatbot và tư vấn.

PHẦN 2: KIẾN TRÚC HỆ THỐNG & ĐỌC CODE (Trọng tâm)

Kiến trúc ứng dụng chia làm 4 tầng chính. Khi thuyết trình đến đâu, bạn hãy mở file tương ứng lên để minh họa.

1. Tầng Model (Định nghĩa cấu trúc dữ liệu)

- Ý nghĩa:** Là các class đại diện cho các đối tượng trong thực tế (Xe máy, Khách hàng, Phiếu sửa chữa).
- Vị trí code:** ... \data\model\
- File cần mở demo:** **Motorcycle.java**
- Cách nói:** "Để quản lý xe, nhóm tạo Model Motorcycle chứa các thông tin cơ bản như id, name, price, quantity. Nó đóng vai trò lưu trữ dữ liệu trong RAM để truyền từ Giao diện xuống Database."

2. Tầng Database (Truy xuất dữ liệu)

- Ý nghĩa:** Cung cấp các thao tác Thêm/Sửa/Xóa/Lọc dữ liệu. Mặc dù báo cáo nhắc tới Room, thực tế code đang ứng dụng Firebase Firestore để thao tác trực tiếp và đồng bộ thời gian thực.
- Cách nói:** "Dữ liệu thay vì lưu cục bộ bằng Room, nhóm đã tích hợp cấu trúc lưu trữ NoSQL trên Firebase Firestore để đảm bảo dữ liệu đồng bộ real-time. Các thao tác Insert/Query được xử lý bất đồng bộ trong các Activity hoặc ViewModel."

3. Tầng Activity & Tầng Xử lý Logic (Điều khiển luồng)

- **Ý nghĩa:** Hứng sự kiện click từ người dùng, gọi Database và điều phối UI.
- **Vị trí code:** `...\ui\inventory\`
- **File cần mở demo:** `AddEditMotorcycleActivity.java`
- **Cách nói:** "Activity là nơi chứa logic chính. Ví dụ ở tính năng *Thêm Xe*, sau khi người dùng nhập đủ Text, Activity sẽ lấy dữ liệu từ `EditText`, khởi tạo Model `Motorcycle`, gọi hàm `insert` vào database, báo Toast thành công và gọi lệnh `finish()` để quay về màn hình trước."

4. Tầng Adapter (Cầu nối giao diện danh sách)

- **Ý nghĩa:** Trích xuất từng thuộc tính của Object đưa lên từng dòng của danh sách RecyclerView.
- **Vị trí code:** `...\ui\inventory\MotorcycleAdapter.java`
- **File cần mở demo:** `MotorcycleAdapter.java`
- **Cách nói:** "Adapter là một thành phần rất quan trọng. Mọi người chú ý 3 hàm chính: `onCreateViewHolder` làm nhiệm vụ nạp file giao diện XML nhỏ (`item_motorcycle`) cho từng dòng; `onBindViewHolder` là nơi đổ dữ liệu (tên xe, giá tiền) vào cái view đó; và `getItemCount` trả về số lượng dòng."

5. Tích hợp Trí tuệ nhân tạo (AI)

- **Vị trí code:** Thư mục `...\ui\ai\`
- **Cách nói:** "Hệ thống tích hợp AI để tư vấn. Khi người dùng nhập text, Activity gửi Request API lên server Gemini. Khi server phản hồi, app sẽ tạo một đối tượng Model `ChatMessage` đánh dấu là của AI, rồi gọi hàm `adapter.notifyDataSetChanged()` để cập nhật lại màn hình chat ngay lập tức."

BÍ KÍP TRÌNH BÀY (Dành cho bạn)

- **Mở code sẵn:** Trước khi lên thuyết trình, hãy mở sẵn các file `Motorcycle.java`, `AddEditMotorcycleActivity.java`, `MotorcycleAdapter.java` trên các tab của Android Studio.
- **Chỉ điểm (Point out):** Đừng đọc code từng dòng. Thay vào đó hãy bôi đen khối code (ví dụ bôi đen hàm `onBindViewHolder` trong Adapter) và giải thích ý nghĩa bằng tiếng Việt.
- **Nói về Data Flow (Luồng dữ liệu):** Luôn chốt lại cách dữ liệu đi: **Nhập UI (Activity) → Tạo Model → Đưa qua Adapter hiển thị hoặc đẩy xuống Database.**

PHẦN 3: VỊ TRÍ CODE CỦA CÁC MODULE QUAN TRỌNG KHÁC

Dưới đây là đường dẫn đến các file xử lý logic cụ thể mà bạn đã hỏi, rất hữu ích khi bạn giám khảo đặt câu hỏi kiểm tra:

1. Module Sửa chữa (Tạo phiếu, AI gợi ý dịch vụ, Fallback)

 **Vị trí code:** Thư mục `...\ui\repair\`

- **Tạo phiếu sửa chữa & Xử lý AI / Fallback:** Mở file `CreateRepairActivity.java`. File này đảm nhận việc nhập thông tin lỗi, gọi API AI để gợi ý dịch vụ (hàm `performAiAnalysis` ở **dòng 195**). Nếu AI không hoạt động (lỗi mạng, hết quota...), hệ thống tự động dùng tính năng **Fallback** (hàm `suggestByKeywords` ở **dòng 256** để tìm dịch vụ qua từ khóa mapping sẵn). Việc lưu phiếu xuống Database xử lý ở hàm `saveRepairOrder` (**dòng 300**).

- **Giao diện hiển thị gợi ý AI:** `AIRepairSuggestionAdapter.java` (để hiển thị danh sách dịch vụ mà AI đề xuất).
- **Quản lý danh mục Dịch vụ (Service):** `RepairServicesFragment.java`.

2. Module Khách hàng (CRUD, Lịch sử, Hồ sơ)

📁 **Vị trí code:** Thư mục `...\ui\customer\`

- **Thêm/Sửa/Xóa (CRUD) danh sách khách:** `CustomerFragment.java` (Hiển thị danh sách và nút Thêm khách hàng ở hàm `showAddCustomerDialog` dòng 99) và `CustomerDetailActivity.java` (Sửa, Xóa và xem thông tin chi tiết).
- **Lịch sử giao dịch của khách:** Nằm trong `CustomerDetailActivity.java` (Load các đơn mua hàng và phiếu sửa chữa của khách đó).
- **Hồ sơ cá nhân (Dành cho Role User):** `UserProfileFragment.java` (Hiển thị thông tin account, hạng thành viên).

3. Module Nhà cung cấp & Đơn nhập hàng

📁 **Vị trí code:** Thư mục `...\ui\supplier\`

- **Quản lý nhà cung cấp (CRUD):** `SupplierFragment.java` (Hiển thị và thao tác thêm sửa xóa nhà cung cấp).
- **Đơn nhập hàng:** `ImportOrderActivity.java` (Xử lý việc nhập thêm xe vào kho từ nhà cung cấp).

4. Hệ thống Tích điểm Thành viên

Hệ thống này không nằm ở một file duy nhất mà là sự kết hợp giữa Dữ liệu và Xử lý thanh toán:

- **Nơi định nghĩa Hạng & Điểm:** Nằm trong Model `...\data\model\Customer.java` (Chứa thuộc tính `memberRank` ở dòng 28 và `loyaltyPoints` ở dòng 29).
- **Nơi tính toán Cộng Điểm:** Khi thanh toán xong một đơn hàng bán xe, logic cộng điểm sẽ chạy. Nằm trong file `...\ui\sales\CreateSaleActivity.java` và gọi logic tính điểm ở `...\viewmodel\SalesViewModel.java` hoặc `CustomerViewModel.java`.
- **Nơi hiển thị:** Mở `CustomerDetailActivity.java` (cho Admin/Sales xem) hoặc `...\ui\dashboard\UserDashboardFragment.java` (cho Khách hàng tự xem hạng của mình).